

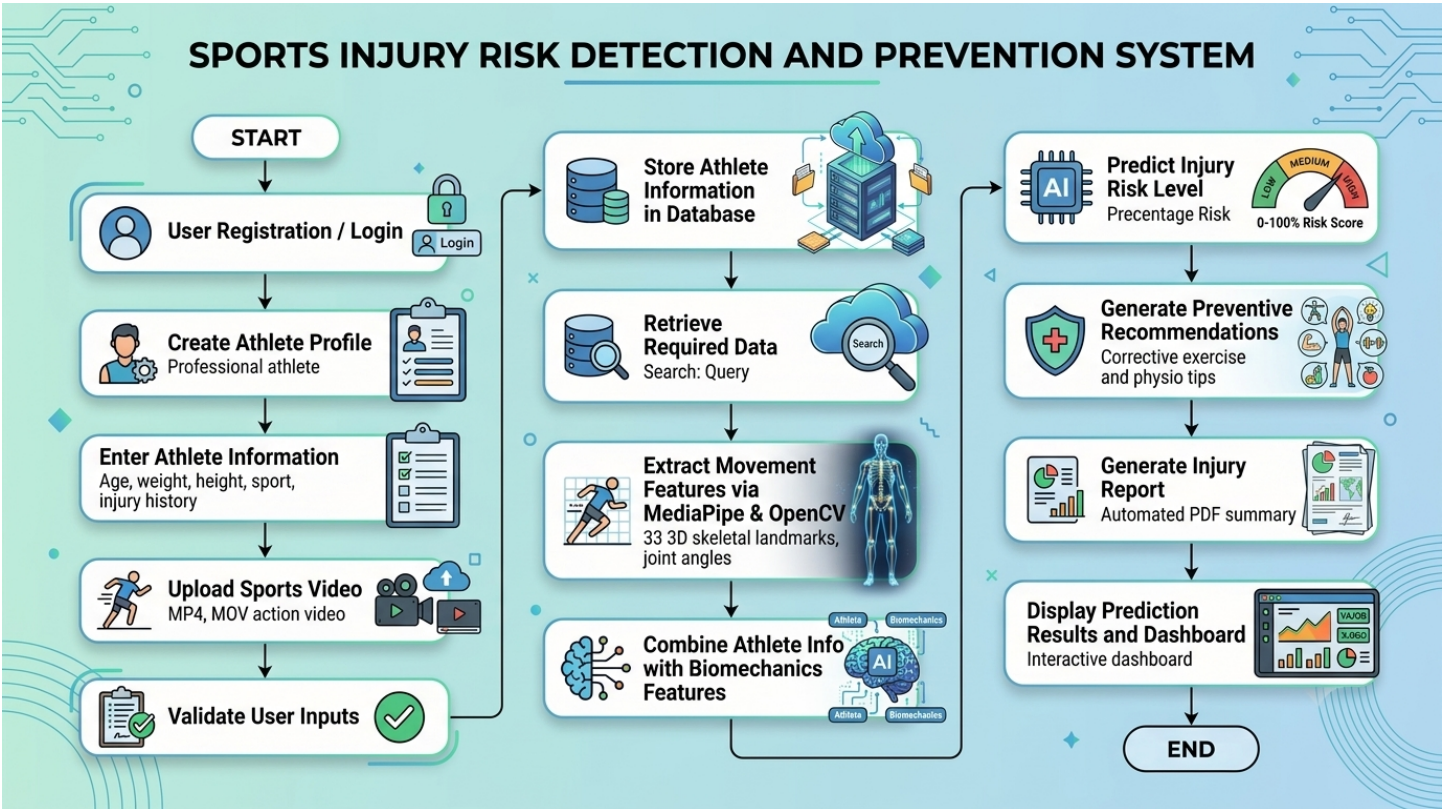
Sports Injury Risk Detection and Prevention System

Comprehensive Technical Architecture, Cloud Hosting, Detailed Workflow & AI Roadmap

1. Executive Overview & Problem Statement

Non-contact sports injuries (such as ACL ruptures, meniscus tears, and lower back strains) are predominantly caused by biomechanical movement flaws (e.g., knee valgus collapse, severe bilateral asymmetry, and excessive trunk lean). The **Sports Injury Risk Detection System** is a sensorless screening platform that transforms standard video into quantifiable 3D kinematic joint coordinates, predicts injury risk probability, and prescribes targeted corrective exercises.

2. System Workflow Infographic



3. Technical Stack Matrix

Component	Technology	Role & Technical Function
Frontend UI	React.js 18 + Vite	Modular Single Page Application (SPA) dashboard, athlete records & video upload interface.
Web Server	Nginx (Alpine)	High-concurrency static asset server (~15MB RAM), SPA fallback routing (`try_files`), gzip compression.
Backend API	FastAPI + Uvicorn	Asynchronous REST API, native OpenAPI Swagger UI (`/docs`), automated Pydantic schema validation.
Pose Tracking AI	Google MediaPipe	Deep learning neural network tracking 33 3D skeletal landmarks per frame with sub-pixel precision.
Video Processing	OpenCV (`cv2`)	Frame decoding, frame sampling rate standardization, visual stick-figure skeletal overlays.
Kinematics Math	NumPy & Pandas	Vector trigonometry, joint angle computation, time-series landmark DataFrames.
Persistence Layer	SQLAlchemy + SQLite	Relational storage for user auth, athlete profiles, video metadata, and risk records.
Report Generator	ReportLab	Automated compilation of styled clinical PDF reports with observations and recommendations.
Containerization	Docker & Compose	Multi-container isolation with persistent storage volumes (`uploads`, `results`, `data`).

4. Cloud Hosting & Production Deployment Architecture

Platform	Component	Scalability & Benefits
Vercel	Frontend (React UI)	Global Edge CDN, automated GitHub CI/CD, zero-maintenance static serving.
Render	Backend (FastAPI)	Docker container web service, persistent disk storage for videos, automated SSL.
Railway	Full Stack Stack	One-click multi-container deployment with internal Docker networking.
AWS / GCP / VPS	Docker Compose	Single-command production deployment (`docker compose up -d`) with full data privacy.

5. Detailed Step-by-Step User & Data Processing Workflow

Phase 1: User Registration & Session Security

Coaches/physios create an account or sign in. Passwords are encrypted with SHA-256 and unique salts. A secure UUID session token is generated in SQLite and attached as Authorization: Bearer <token> to all subsequent API requests.

Phase 2: Athlete Profile & Baseline Recording

User registers athlete demographic data: Full Name, Sport, Age, Height (cm), Weight (kg), and Prior Injury History. This provides historical context for kinetic load calculations.

Phase 3: Video Ingestion & Validation

User selects the athlete, chooses activity (Squat, Running, Jumping/Landing), and uploads video (MP4/MOV). Backend validates MIME types, headers, generates a collision-free UUID, and writes to /app/uploads/.

Phase 4: AI Computer Vision & Pose Tracking

OpenCV decodes frames; Google MediaPipe deep neural network extracts **33 3D coordinates** \$(x,y,z)\$ per frame. An annotated video with drawn skeletal landmarks is saved to /app/results/.

Phase 5: Biomechanical Kinematic Calculations

Vector trigonometry computes Knee & Hip Range of Motion (ROM), Bilateral Symmetry Deficit % (comparing left vs right leg loading), Trunk Lean Angle, and Movement Consistency across repetitions.

Phase 6: Injury Risk Prediction & Clinical Scoring

A composite weighted model computes a 0-100% Risk Score classified into **LOW RISK (0-33%)**, **MEDIUM RISK (34-66%)**, or **HIGH RISK (67-100%)**, identifying specific flaws like knee valgus or trunk shear.

Phase 7: Dashboard Analytics & PDF Report Export

Results populate the live dashboard in real-time. A formal medical PDF report with charts, observations, and targeted corrective exercises is compiled via ReportLab for download.

6. Key System Benchmarks & Technical Metrics

Metric / Benchmark	Performance Value	Description
Pose Keypoints	33 3D Landmarks	Full-body tracking: shoulders, elbows, wrists, hips, knees, ankles, heels, toes.
Processing Speed	25 – 45 FPS (CPU)	Real-time / near-real-time CPU execution without requiring dedicated GPUs.
Resolution Support	720p / 1080p / 4K	Standardized scaling during OpenCV decoding pipeline.
API Latency	< 45 ms (CRUD)	Asynchronous non-blocking endpoints via FastAPI.
Supported Formats	MP4, MOV, AVI, WebM	Cross-platform desktop and mobile video compatibility.

7. Future AI/ML Roadmap: Supervised Injury Prediction (Milestone 3)

In Milestone 3, the rule-based risk evaluation will be upgraded to a **Supervised Machine Learning & Deep Learning model**. The Milestone 2 feature extraction pipeline directly supplies the exact numerical vectors needed for ML training:

A. Machine Learning Feature Vector:

- Knee Symmetry Deficit (%): Left vs right knee angle deviation during impact
- Hip Symmetry Deficit (%): Pelvic tilt and bilateral hip displacement
- Max Trunk Lean Angle (°): Correlates with spinal shear stress and core fatigue
- Movement Quality Score (0-100): Composite score of joint coordination
- Athlete Baseline (Age, Weight, Sport): Physical demographics and body levers
- Reported Injury History (0 or 1): Historical risk factor weighting

B. Recommended AI Frameworks:

1. **Tabular Machine Learning (XGBoost / LightGBM / Random Forest):** High accuracy, low latency, and robust interpretability for vector-based injury prediction.
2. **Temporal Deep Learning (PyTorch LSTM / 1D-CNN):** Directly models raw time-series joint angle sequences over the full duration of athletic jumps and sprints.
3. **Explainable AI (SHAP / LIME):** Quantifies exactly why an athlete was flagged (e.g. *'+35% risk attributed to 24° left knee valgus collapse'*).

C. Model Integration:

The trained model (injury_model.pkl) will be stored in backend/app/models/ and loaded once on startup. The existing compute_risk() function will call model.predict_proba() seamlessly with zero breaking changes to frontend consumers.